Low-Level Design (LLD): RAG-Based Customer Support Assistant

1. Module-Level Design

The system is divided into five modular Python scripts to ensure separation of concerns and maintainability.

- **Setup Module (`config.py`):** Centralizes environment variables and initializes the LLM (`ChatGoogleGenerativeAI`) and the Embedding Model (`HuggingFaceEmbeddings`).
- **Document Processing & Chunking Module (`document_processor.py`):** Handles the offline data ingestion. Utilizes `PyPDFLoader` to read the document and `RecursiveCharacterTextSplitter` to enforce a strictly defined chunking strategy.
- **Vector Storage & Retrieval Module (`retriever.py`):** Encapsulates the ChromaDB connection. Defines a specific search function that executes a similarity search against the vector database based on the query.
- **Graph Execution & HITL Module (`graph_workflow.py`):** The core orchestrator. Contains the LangGraph implementation, node definitions, conditional routing criteria, and the Human-in-the-Loop pause mechanism.
- **Interface Module (`app.py`):** The Command Line Interface (CLI) that manages the continuous `while` loop, captures user inputs, passes them to the compiled graph, and prints the final state responses.

2. Data Structures

Document and Chunk Representation: Documents and chunks are managed using LangChain's native `Document` object schema:

JSON
```
{
  "page_content": "String containing the 500-character text chunk.",
  "metadata": {
    "source": "knowledge_base/support_policy.pdf",
    "page": 0
  }
}
```

State Object for Graph Workflow (LangGraph): Data flowing between nodes is strictly typed using a Python `TypedDict`. This ensures every node has access to the exact same conversational state parameters.

Python
```
class AgentState(TypedDict):
    query: str          # The original user input
    context: str        # The retrieved text chunks from ChromaDB
    response: str       # The drafted AI answer or final human resolution
    needs_human: bool   # The boolean flag triggering the HITL routing
```

3. Workflow Design (LangGraph)

The graph is designed as a state machine with strict directional and conditional edges.

Nodes:

1. `retrieve_node(state)`: Takes `state["query"]`, queries ChromaDB, and mutates `state["context"]`.
2. `generate_node(state)`: Passes `state["query"]` and `state["context"]` to the LLM. Mutates `state["response"]` and evaluates the `needs_human` flag.
3. `human_intervention_node(state)`: A specialized node that suspends graph execution to accept terminal input, mutating `state["response"]` with the human's manual answer.

Edges:

- `START` → `retrieve_node`
- `retrieve_node` → `generate_node`
- `generate_node` → *Conditional Routing*
- `human_intervention_node` → `END`

4. Conditional Routing Logic

Routing decisions are controlled primarily through Prompt Engineering inside the `generate_node`. The LLM is instructed with two strict rules:

1. **Answer Generation Criteria:** If the exact answer exists within the retrieved context chunks, the LLM generates a standard text response, and `needs_human` remains `False`.
2. **Escalation Criteria:** If the retrieved context does not contain the answer, or if the question is deemed highly complex, the LLM is instructed to output a specific trigger word: `"ESCALATE"`.
3. **Graph Interpretation:** The `generate_node` parses the LLM's raw output. If the string `"ESCALATE"` is detected, the node sets `state["needs_human"] = True`. The `route_after_generation` edge checks this boolean to decide whether to route to the `human_intervention_node` or terminate the graph.

5. HITL (Human-in-the-Loop) Design

The HITL mechanism leverages the synchronous nature of Python's built-in `input()` function within the LangGraph architecture.

- **When Escalation is Triggered:** The graph enters the `human_intervention_node`. A visual warning block is printed to the terminal, displaying the user's original query.
- **Execution Pause:** The code executes `input("Agent, please type the manual response: ")`. This fundamentally halts the graph's execution thread, preventing any further data flow.

- **Integration:** Once the human types the answer and presses Enter, the string is prepended with `[Human Agent]` and saved to `state["response"]`. The graph resumes and immediately routes to `END`, passing the human's answer back to the user interface.

6. API / Interface Design

The application uses an interactive Command Line Interface (CLI) to simulate an API request/response cycle.

- **Input Format:** Standard raw text string captured via the terminal.
- **Output Format:** Formatted text string printed to the terminal, prefixed by "Assistant:" or "Assistant: [Human Agent]" to explicitly denote the source of the answer.
- **Interaction Flow:** A continuous `while True` loop that persists until the user inputs an exit command (`"quit"`, `"exit"`). Every new query instantiates a fresh `AgentState` dictionary, ensuring state isolation between distinct questions.

7. Error Handling

- **Missing Data / File Errors:** The `document_processor.py` implements `try/except` blocks during the `PyPDFLoader` execution. If the PDF is missing, it catches the `FileNotFoundError` and alerts the developer rather than crashing the ingestion pipeline.
- **No Relevant Chunks Found:** If ChromaDB returns blank context, the `generate_node` prompt logic prevents hallucinations by defaulting to the "ESCALATE" command.
- **LLM / Network Failure:** The `app.py` execution loop wraps the `app.invoke(state)` call in a global `try/except` block. If the Gemini API times out or fails, the interface gracefully catches the exception, prints a `[System Error]`, and keeps the application running for the next query.